浙江大学实验报告

专业：电子科学与技术
姓名：郑智瑄，徐棵
学号：3230103913,3230104003
日期：April 30, 2025
地点：东 4-421

课程名称：电子工程训练（甲）　指导老师：李培弘　实验类型：测试型
实验名称：电子工训智能插座 DIY 项目　成　绩：　签　名：

装 订 线

一、 电装

1. 电装元器件顺序

为方便焊接，我们采取了先低后高的焊接顺序，首先焊接了最低的电阻与二极管 1N4148，然后焊接较低的瓷片电容、继电器插座/芯片插座、Type-C USB 插口、发光二极管与三极管，接着焊接了较高的按钮、电解电容、温度传感器插座、可变电阻、正面排针，最后焊接了最高的双联 USB 接口和自锁开关

2. 供电电路测试

① Type-C 插上外接 5V 电源，观察“D3”是否亮起

② 用万用表的直流电压档，检查 J1 排针上的第 3 脚（标注 5V）与 4 或 5 脚（标注 GND）之间的 5V 电压是否正确

③ 用万用表的直流电压档测量 USB 电压是否与外接 5V 电源电压一致，即约 5V。电压表的负极表棒接电路系统的地线端 GND（TP3）

检测项目	检测结果
电源空载时的输出电压：	5.110V
插上电源后D3状态（亮/灭）：	亮
插上电源后，标注5V处（J1的3脚）的电压（即以万用表直流电压档测量标注5V处对地线GND的电压）：	5.109V

Figure 1: 供电电路测试结果

【结果分析】：供电电路无问题，能正常运行

3. USB 插座供电控制电路

(1) USB2 信号路径分析

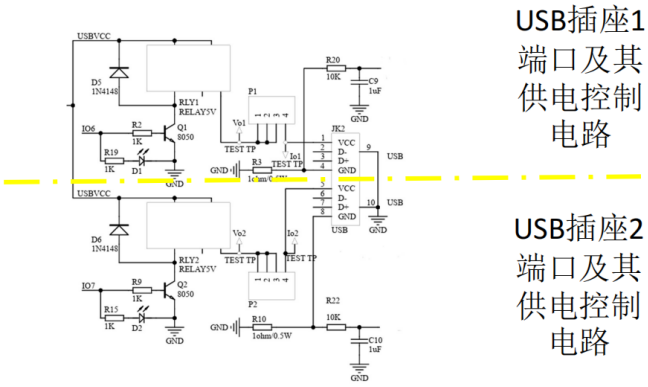


Figure 2: USB 插座供电控制电路

【USB2 信号路径分析】：引脚 IO7 通过高/低电平（即 +5V/0V），可控制三极管 Q2 的导通/截止，从而决定继电器 RLY2 的通断，最终决定 USB 供电插座 JK2 上的 USB2 口是否有 5V（即 USBVCC）电源输出。供电电流将从 USB2 口的 VCC 引脚流出，经过外接用电器，从 USB2 口的 GND 流回。

(2) USB 插座供电控制电路测试

① 测试插座供电输出控制功能是否完备：首先连接一根杜邦线到 J1 排针的第 3 脚（即 5V）

② 测试 USB 供电插座 JK2 的 USB1 口的输出控制：将杜邦线另一端连接到排针 J3 的第 7 脚（即 L1）

③ 测试 USB 供电插座 JK2 的 USB2 口的输出控制：将原先连接 J3 排针第 7 脚的杜邦线一端连接到 J3 排针的第 8 脚（即 L2）

检测项目	检测结果
以杜邦线连接标注L1处（J3的7脚）至标注5V（J1的3脚），观察到的现象：	D1（亮/灭）_亮_，继电器RLY1（吸合/断开）_吸合_，USB供电插座1的供电电压（即P1）_5.055_V。
以杜邦线连接标注L1处（J3的7脚）至标注GND（J1的4或5脚），观察到的现象：	D1（亮/灭）_灭_，继电器RLY1（吸合/断开）_断开_，USB供电插座1的供电电压（即P1）_0_V。
以杜邦线连接标注L2处（J3的8脚）至标注5V（J1的3脚），观察到的现象：	D2（亮/灭）_亮_，继电器RLY2（吸合/断开）_吸合_，USB供电插座2的供电电压（即P2）_5.056_V。
以杜邦线连接标注L2处（J3的8脚）至标注GND（J1的4或5脚），观察到的现象：	D2（亮/灭）_灭_，继电器RLY2（吸合/断开）_断开_，USB供电插座2的供电电压（即P2）_0_V。

Figure 3: USB 插座供电控制电路测试结果

【结果分析】：USB 插座供电控制电路无问题，能正常运行

4. 指示灯与按钮电路测试

检测项目	检测结果
电路上电，以杜邦线连接标注L4处（J4的1脚）至标注5V（J1的3脚），观察到的现象：	D4（亮/灭）亮。 这里通过标注4所施加的5V电压，就是后续系统能通过软件输出的二进制控制信号“1”。
电路上电，以杜邦线连接标注L4处（J4的1脚）至标注GND（J1的4或5脚），观察到的现象：	D4（亮/灭）灭。 这里通过标注4所施加的0V电压，就是后续系统能通过软件输出的二进制控制信号“0”。

Figure 4: 指示灯测试结果

【结果分析】：指示灯控制电路无问题，能正常运行

5. 电装总体完成后的初步通电测试

① 先不安装芯片 LM324AN 和 LM35DZ，通过接入外接 5V 直流稳压电源，进行初步的通电测试。

② 检查 U1（即 LM324AN）芯片插座上的电源电压（即该插座上 4 脚和 11 脚之间的电压）是否为正常的 5V，并记录检测结果。

③ 检查 U2（即 LM35DZ）芯片插座上的电源电压是否为正常的 5V（即该插座上 1 脚和 3 脚之间的电压），并记录检测结果。断开外接 5V 直流稳压电源。

④ 上述电压均检测无误后，将芯片 LM324AN 和芯片 LM35DZ 安插到其相应的插座上。

检测项目	检测结果
LM324AN的电源电压（V）：	5.108V
LM35DZ的电源电压（V）：	5.108V

Figure 5: 集成电路芯片的供电检测结果

【结果分析】：集成电路芯片的供电无问题，能正常运行

二、 调试

1. 基于硬件调整的温度值标定调试

【实验步骤】：电路中的可变电阻 VR1 用于调整温度测量值输入时的转换比例，所以需要根据在测试程序中显示的温度值，同时参考当前室温（通过我们教室中安放的温度计来读取），并通过调整可变电阻 VR1 的螺丝旋钮来调整显示值，并使之与实际室温的尽量保持一致

项目	测量值
当前实际室温（摄氏度）	31.4
经你完成调试后测试程序显示温度（摄氏度）	31.520
是存在严重的元器件离散性问题？（是/否）	否

Figure 6: 供电电路测试结果

【结果分析】：无明显元器件离散问题，可与实际室温拟合

2. 基于软件后处理的电流数据标定

(1) 采样电流数据

Val（串口监视器读取电流值）	I（P1、2处测得的实际电流值）
483mA	80.8mA
2700mA	467.1mA

Figure 7: 供电电路测试结果

(2) 标定公式

计算求解 k,b, 最终的标定公式为：

$$I = 0.174Val–3.242$$

```
// send sensor values:
Serial.print("Socket #1 current: ");
Serial.print((float)Sensor0*0.174-3.242);
Serial.print(" mA");
Serial.print("\t\t");
Serial.print("Socket #2 current: ");
Serial.print((float)Sensor1*0.173-2.42);
Serial.println(" mA");
```

Figure 8: 修改后代码段

(3) 数据采集与误差分析

用你标定好的小程序，采集量程范围内的 10 个值，并比较其与电流表实际读数的差异

Val（串口监视器读取电流值）/mA	I（P1、2 处测得的实际电流值）/mA	误差
80.63	80.8	0.21%
118.56	118.8	0.20%
162.06	162.5	0.27%
213.56	214	0.21%
238.27	237.8	0.20%
260.89	260.6	0.11%
278.29	277.5	0.28%
322.49	321.2	0.40%
353.98	353.7	0.08%
410.18	411.4	0.30%
467.25	467.5	0.05%

Table 1: CH1 数据采集与误差分析

(4)　另一路电流标定

同上进行电流标定，结果如下：

$$I = 0.173Val–2.24$$

Val（串口监视器读取电流值）/mA	I（P1、2 处测得的实际电流值）/mA	误差
80.62	80.8	0.22%
105.53	105.9	0.35%
217.29	217.9	0.28%
249.47	249.2	0.11%
274.03	273.9	0.05%
303.44	303.1	0.11%
334.76	334.8	0.01%
378.7	378.7	0.00%
406.03	407	0.24%
426.1	427.7	0.37%
469.87	470.4	0.11%

Table 2: CH2 数据采集与误差分析

【结果分析】：经标定后，串口监视器读得两路电流与实际电流值拟合较好，误差较小

3. 软硬件相结合的系统调试

(1)　测试任务一

【实验步骤】：利用小台灯和小程序 $SmtScktTest_USB$，确认程序控制 USB 插座中 1# 和 2# 的插座分别由哪个引脚控制

输出命令	1#和2#插座的状态（通/断）	控制电压（用电压表测量L1或L2）
发送字符A	智能插座1#插座的输出电压，P1的电压：5.096 V	J3的 L1（选L1或L2，即你程序中输出控制信号的那个引脚）脚电压：2.944 V
发送字符a	智能插座1#插座的输出电压，P1的电压：0.001 V	J3的 L1（选L1或L2，即你程序中输出控制信号的那个引脚）脚电压：0.001 V
发送字符B	智能插座2#插座的输出电压，P2的电压：5.093 V	J3的 L2（选L1或L2，即你程序中输出控制信号的那个引脚）脚电压：2.941 V
发送字符b	智能插座2#插座的输出电压，P2的电压：0.001 V	J3的 L2（选L1或L2，即你程序中输出控制信号的那个引脚）脚电压：0.000 V

Figure 9: 测试任务一测试结果

【结果分析】：1# 插座由“A”和“a”控制开关，2# 插座由“B”和“b”控制开关

(2)　测试任务二

【实验步骤】：利用各类用电器，测量其在智能插座使用中的各类测量值

项目	测量值	单位
1#插座外接的用电器名称	小台灯	
1#插座电流值	187.46	mA
2#插座外接的用电器名称	可调光台灯	
2#插座电流值（最亮时）	260.19	mA
2#插座电流值（最暗时）	41.18	mA
插座电压值	5.054	V
温度测量值（环境温度）	31.520	° C
温度测量值（手指触碰温度）	33.321	° C

Figure 10: 测试任务二测试结果

(3)　测试任务三

【实验步骤】：将 2# 插座上的可调光台灯换成风扇，并接通风扇的插座电源，测量风扇的各档工作电流值。

项目	测量值	单位
风扇慢速档电流值	203.44	mA
风扇中速档电流值	317.58	mA
风扇快速档电流值	380.08	mA

Figure 11: 测试任务三测试结果

【实验异常分析】：上述测试任务（2）、（3）中有哪些测量值有异常？利用示波器观察并拍照记录出现这些异常时输出供电电压的波形，说明其产生的原因。

装 订 线

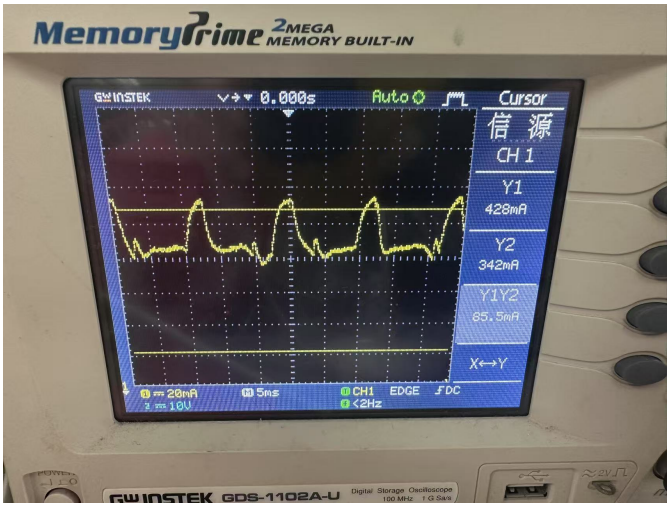


Figure 12: 异常输出供电电压波形

【异常值】：任务三中测量风扇的各档工作电流值时电流值约有 ± 20mA 的波动
【分析】：小风扇由 PWM 驱动，PWM 为脉冲电压信号，电流会随着脉冲的高低电平变化而周期性变动

4. 软硬件联调任务一

【实验要求】：在我们的智能插座给风扇供电时，会出现电流值不停地大小波动变化的现象，这是由于该风扇的调速采用了方式 PWM 的控制请在 *SmtScktTest_USB.ino* 程序中添加一个子程序，通过调用该子程序，实现对采集的电流值取平均值，以便使同一转速挡位的电流值显示趋于平稳。
【提示】：在取平均值时，可以选择给以往值和当前值取不同的权值，即 加权平均法；也可以选择 移动平均法 等，来调整平均值的计算效果，以达到既能在风扇档位变化时，尽快地响应电流的变化，同时又能在各档位保持不变时，使电流的显示值趋于平稳。
【代码与实现方式】：我们选择了 移动平均法 来使电流显示值平稳，*movingAverageFilter* 函数可对输入的电流值用移动平均法进行滤波，其操作过程如下：

在读取原始电流值后，通过参数传递原始值、样本数组、索引和累加和，在函数中，通过滑动窗口动态更新平均值，采用环形缓冲区索引管理来移除最旧值，添加最新值，最后返回窗口内数组的平均值

本次实验中，通过定义宏窗口大小 WINDOW_SIZE 来平衡尽快响应电流变化，与电流的显示值平稳的要求，当提高 WINDOW_SIZE 时，电流的显示值趋于平稳，但响应变慢，当减小 WINDOW_SIZE 时，响应变快，但显示值波动较大，经过实验测试，最终发现当 WINDOW_SIZE 为 8 时，电流显示值较为平稳且响应较快，故最终取 WINDOW_SIZE 为 8

```
#define WINDOW_SIZE 8

// 电流通道0
unsigned int samples0[WINDOW_SIZE] = {0};
int index0 = 0;
unsigned long sum0 = 0;

// 电流通道1
unsigned int samples1[WINDOW_SIZE] = {0};
int index1 = 0;
unsigned long sum1 = 0;

unsigned int movingAverageFilter(unsigned int rawValue, unsigned int samples[], int &idx, unsigned long &sum)
  sum -= samples[idx];       // 移除最旧值
  sum += rawValue;           // 添加最新值
  samples[idx] = rawValue;  // 更新样本数组
  idx = (idx + 1) % WINDOW_SIZE; // 环形索引
  return sum / WINDOW_SIZE; // 返回平均值
}
```

Figure 13: 软硬件联调任务一代码段

```
void loop()
{
  // read analog inputs:
  Sensor0 = movingAverageFilter(getCurrent(0),samples0,index0,sum0);
  Sensor1 = movingAverageFilter(getCurrent(1),samples1,index1,sum1);
  Sensor2 = getVoltage();
  Sensor3 = getTemperature();
```

Figure 14: 软硬件联调任务一代码段

5. 软硬件联调任务四

(1) 步骤 1

【实验要求】：首先弄清按下按钮/ 松开按钮时，Arduino 程序分别读取到的按键状态值（二进制的 0/1）。在按钮保持在按下或松开时，按下‘k’命令，以便确定按钮被按下时，被读取到的按钮状态值值是 Arduino 程序 0 还是 1？

检测项目	检测结果	s1按钮的程序读取值(0/1)
电路上电，按下S1按钮并保持，测量S1（J4的4脚）电平（即该点的对地电压）：	S1脚的电平0.000V。 这个就是你按下按钮状态下，系统能通过软件检测到的信号。	0
电路上电，松开S1按钮，测量S1（J4的4脚）电平（即该点的对地电压）：	S1脚的电平3.334V。 这个就是你松开按钮状态下，系统能通过软件检测到的信号。	1

Figure 15: 软硬件联调任务四表

(2)　步骤 2

【实验要求】：采用各种按键速度和连续按键次数，测试程序是否能每次都正确记录按钮次数？如果不是（即存在误读次数）通过示波器查看并记录按钮按动时的波形，并分析误读产生的原因

【示波器显示结果】：改变按键速度与连续按键次数，可能由于此按钮机械性能较好，各种情况下毛刺数量并没有很多，一般均得到左图类似图片，多次尝试才得到如下右图毛刺图片

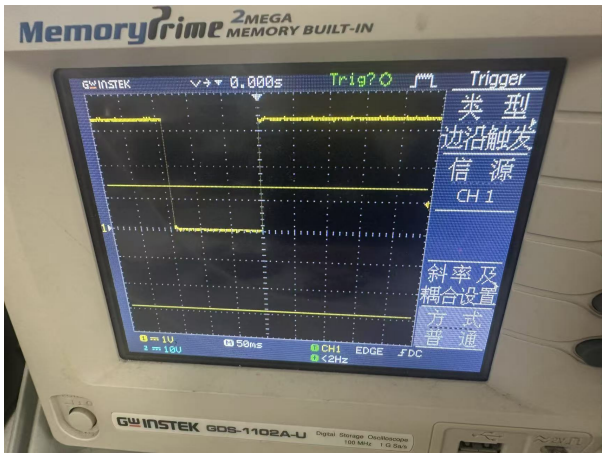


Figure 16: 无毛刺按压结果

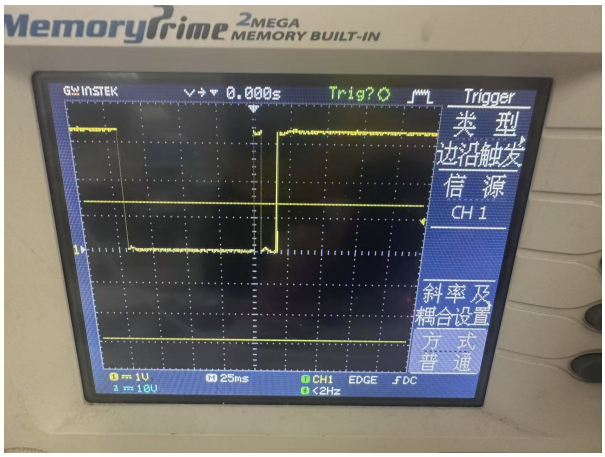


Figure 17: 有毛刺按压结果

(3)　步骤 3

【实验要求】：修改该程序 key_in.ino ，使修改后的程序能准确地读取并记录下以各种按键速度连续按键的次数（注：每“按下放开”只能算一次按键）

【代码段与实验结果】：如图，经修改后，程序能准确地读取并记录下以各种按键速度连续按键的次数，虽然此时还未处理毛刺问题，但或许由于按钮机械性能较好，对各种速度/连续按键处理已然较好

```
void loop()
{
  keyin = digitalRead(keyPin11);
  // get s1 input
  if(keyin != lastkey)
  {
    if(keyin == LOW){
    acc++;
    }
    lastkey = keyin;
  }
```

Figure 18: 软硬件联调任务四代码

(4)　步骤 4

【实验要求】：毛刺由于按键的机械特性而存在。会造成键被误按。常规的去除毛刺方法是在产生毛刺其间暂停对按键信号的读入，这可以用恰当的软件延时来实现。但软件延时过量又会造成对按键响应的迟钝。调试软件使其既能避免毛刺，又能对按键由恰当的及时正确的反应，是一个好键盘的终极最求，需要在按键系统调试过程中结合硬件特性，对软件仔细调教

【代码段与实验结果】：通过在 loop() 中添加消抖函数，忽略小于宏 DEBOUNCE_DELAY 的毛刺，实现对毛刺的过滤，同时增加 [DEBUG] 输出，使得不必每一次都通过“r”获取次数，更加直观明了

```
#define DEBOUNCE_DELAY 50    // 消抖时间阈值（单位：ms）

unsigned long lastDebounceTime = 0;
bool keyPressed = false;
```

Figure 19: 软硬件联调任务四代码

```
void loop()
{
  keyin = digitalRead(keyPin11);
  if (keyin != lastkey) {
    lastDebounceTime = millis();
    lastkey = keyin;
  }

  if ((millis() - lastDebounceTime) > DEBOUNCE_DELAY) {
    if (lastkey == LOW) {  // 按下动作
      keyPressed = true;
    } else {
      if (keyPressed) {    // 释放动作且之前有有效按压
        acc++;
        keyPressed = false;
        Serial.print("[DEBUG] press count: ");
        Serial.println(acc);
      }
    }
  }
```

Figure 20: 软硬件联调任务四代码

三、 系统测试

1. 连接手机

【测试用例描述】:

① 使用安卓手机，关闭移动网络流量，开放热点 (热点名, 密码由英文字母和数字组成)，AP 频段设置在 2.4GHz 频段上

② 在程序中设置 ssid 和 password 与手机热点相符，将程序烧入板子中

③ 运行智能插座小程序，使得手机与智能插座连接

【测试目的】: 验证智能插座可连一个手机
【测试过程及其结果】: 手机与智能插座连接成功

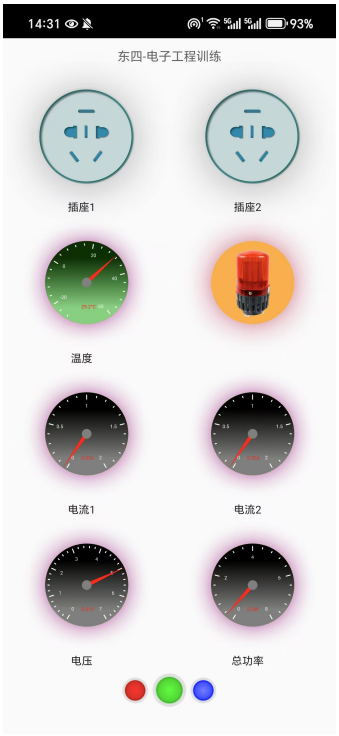


Figure 21: 连接手机成功

2. 插座开关手动控制，以及温度、电压、电流和功率显示

【测试用例描述】

① 将小台灯插在插座 1 上

② 手动打开插座 1，观察继电器与发光二极管 D1 状态

③ 在智能插座 App 读取对应的温度、电压、电流和功率

④ 将风扇插到插座 2 上，并更换不同的挡位，重复步骤（2）（3）

【测试目的】：验证插座开关手动控制，以及温度、电压、电流和功率显示功能
【测试过程及其结果】：插座 1 与插座 2 通路均能正常连通，继电器发出咔哒声响闭合，发光二极管亮起
【附图】：

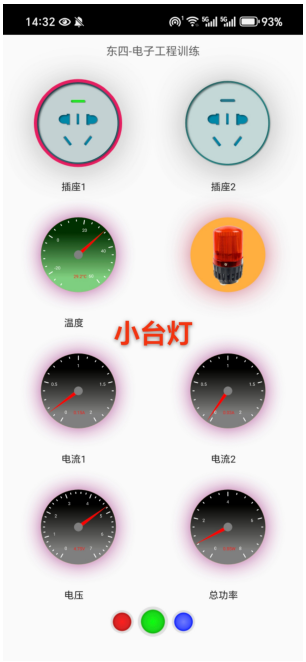


Figure 22: 小台灯

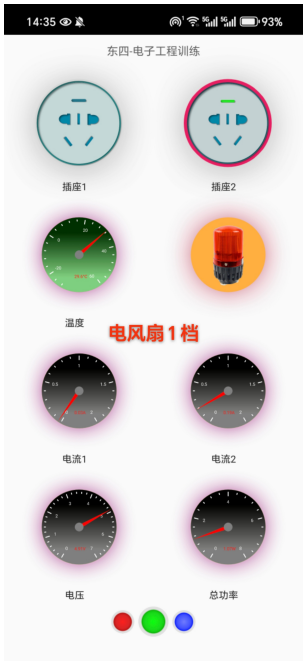


Figure 23: 电风扇 1 档

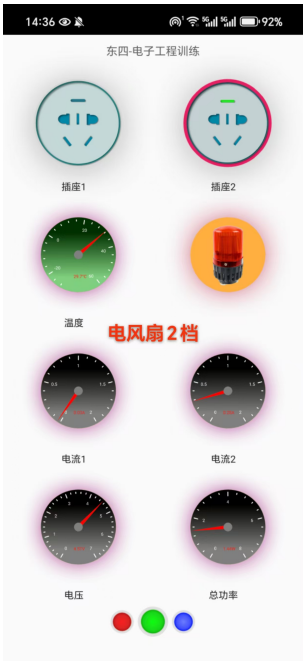


Figure 24: 电风扇 2 档

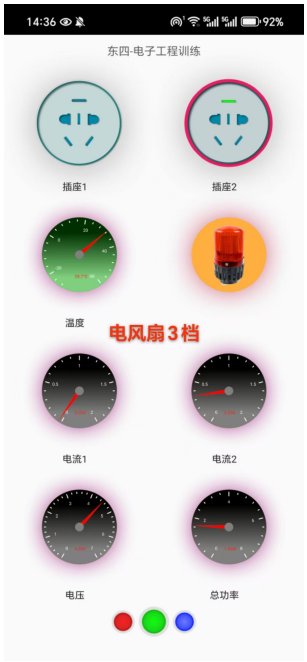


Figure 25: 电风扇 3 档

3. 定时开/关，延时开/关

(1) 定时开关

【测试用例描述】:

① 将小台灯插在插座 1 上，关闭插座 1 电源

② 14：40 在智能插座 App 上设置定时 14：41 开，预期 14：41 小台灯亮起

③ 14：41 在智能插座 App 上设置定时 14：42 关，预期 14：42 小台灯熄灭

【测试目的】: 验证插座开关定时开/关功能
【测试过程及其结果】: 结果符合测试预期，小台灯在 14：41 亮起，在 14：42 熄灭，插座开关定时开/关功能良好
【发现 bug】: 定时界面无法设置秒数
【附图】:

(2) 延时开关

【测试用例描述】:

① 将小台灯插在插座 1 上，关闭插座 1 电源

② 14：45 在智能插座 App 上设置延时 1min 开，预期 14：46 小台灯亮起

③ 14：47 在智能插座 App 上设置延时 1min 关，预期 14：48 小台灯熄灭

【测试目的】: 验证插座开关延时开/关功能
【测试过程及其结果】: 结果符合测试预期，小台灯在 14：46 亮起，在 14：48 熄灭，插座开关延时开/关功能良好
【发现 bug】: 延时界面无法设置秒数

【附图】:



Figure 26: 定时开关



Figure 27: 延时开关

4. 超/欠电压、超电流、超功率、高温/低温告警并自动断开插座供电

(1) 超电压

【测试用例描述】:

① 将风扇插在插座 1 上，关闭插座 1 电源

② 将电压上限设定为 4.8V

③ 开启插座 1 电源，启动风扇 1 挡位，预期超电压告警

【测试目的】: 测试插座开关超电压报警功能
【测试过程及其结果】: 结果符合测试预期，实时电压在超过 4.8V 时瞬间报警，小程序显示电压故障，插座断电，插座开关超电压报警功能良好

(2) 欠电压

【测试用例描述】:

① 将风扇插在插座 1 上，关闭插座 1 电源

② 将电压下限设定为 4.5V

③ 开启插座 1 电源，启动风扇 3 挡，预期欠电压告警

【测试目的】: 测试插座开关欠电压报警功能
【测试过程及其结果】: 结果符合测试预期，实时电压在低于 4.5V 时瞬间报警，小程序显示电压故障，插座断电，插座开关欠电压报警功能良好

(3)　超电流

【测试用例描述】:

① 将风扇插在插座 1 上，关闭插座 1 电源

② 将电流上限设定为 0.2A

③ 开启插座 1 电源，启动风扇 3 挡，预期超电流告警

【测试目的】：测试插座开关超电流报警功能
【测试过程及其结果】：结果符合测试预期，实时电流在超过 0.2A 时瞬间报警，小程序显示电流超限，插座断电，插座开关超电流报警功能良好
【附图】:

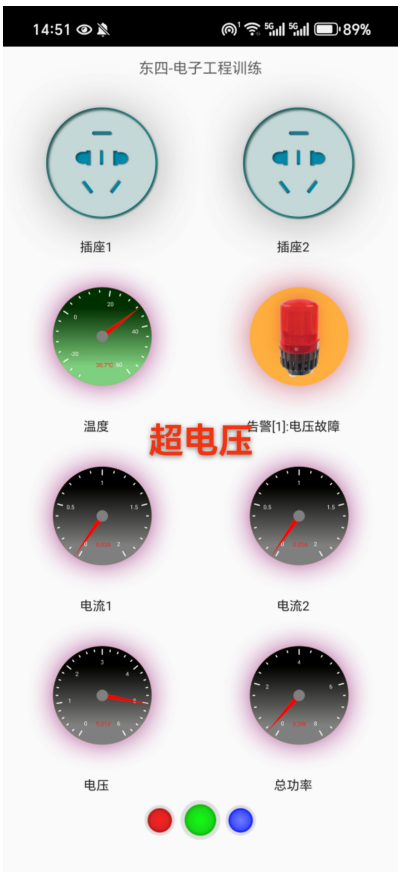


Figure 28: 超电压

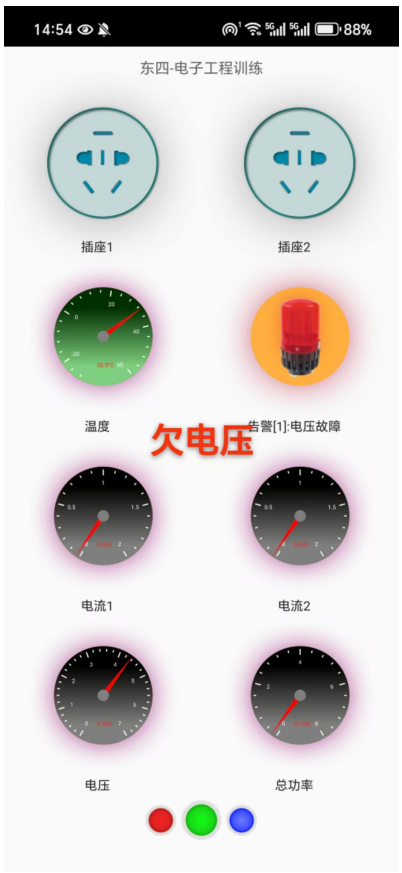


Figure 29: 欠电压

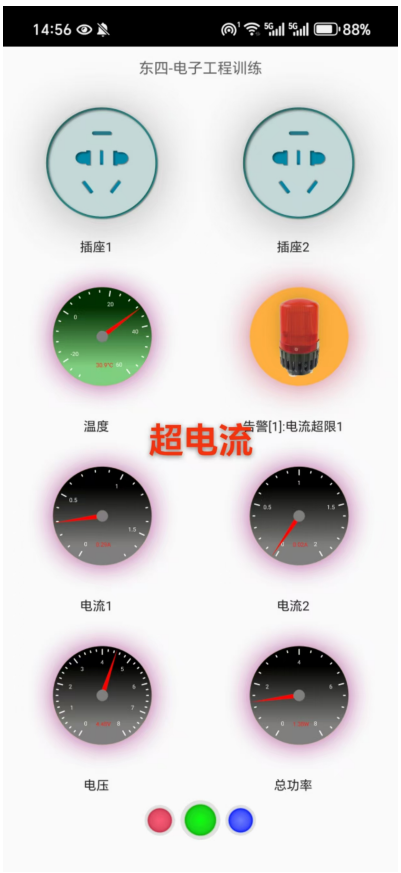


Figure 30: 超电流

(4)　超功率

【测试用例描述】:

① 将风扇插在插座 1 上，关闭插座 1 电源

② 将电流上限设定为 1W

③ 开启插座 1 电源，启动风扇 3 挡，预期超电流告警

【测试目的】：测试插座开关超功率报警功能

【测试过程及其结果】：结果符合测试预期，实时显示功率在超过 1w 时瞬间报警，小程序显示功率超限，插座断电，插座开关超功率报警功能良好

(5) 高温

【测试用例描述】：

① 将风扇插在插座 1 上，关闭插座 1 电源

② 将温度上限设定为 $32°C$

③ 开启插座 1 电源，将手指贴在温度传感器上，预期温度超过 $32°C$ 时高温告警

【测试目的】：测试插座开关高温报警功能

【测试过程及其结果】：结果符合测试预期，实时显示温度在超过 $32°C$ 时瞬间报警，小程序显示温度异常，插座断电，插座开关高温报警功能良好

(6) 低温

① 将风扇插在插座 1 上，关闭插座 1 电源

② 将温度下限设定为 $30°C$

③ 开启插座 1 电源，用小风扇吹温度传感器使之报警，预期温度低于 $30°C$ 时低温告警

【测试目的】：测试插座开关低温报警功能

【测试过程及其结果】：结果符合测试预期，实时显示温度在低于 $30°C$ 时瞬间报警，小程序显示温度异常，插座断电，插座开关低温报警功能良好

【附图】：

装 订 线

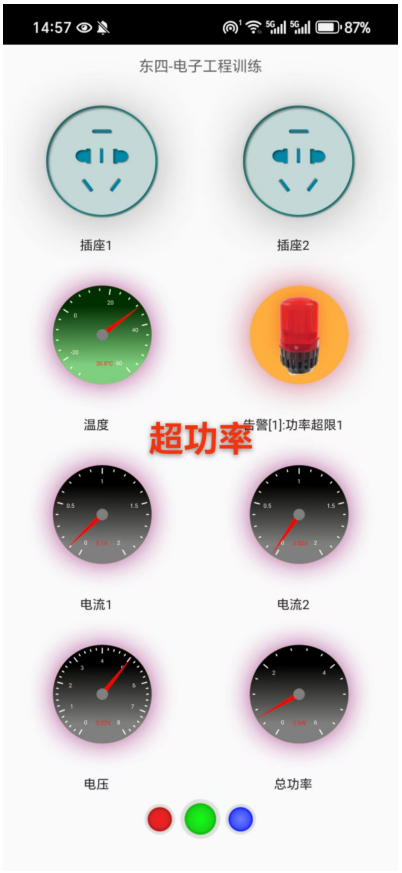


Figure 31: 超功率

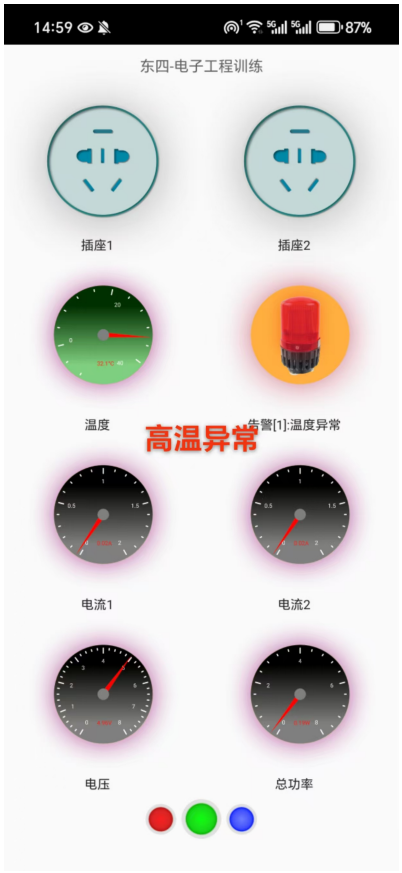


Figure 32: 高温

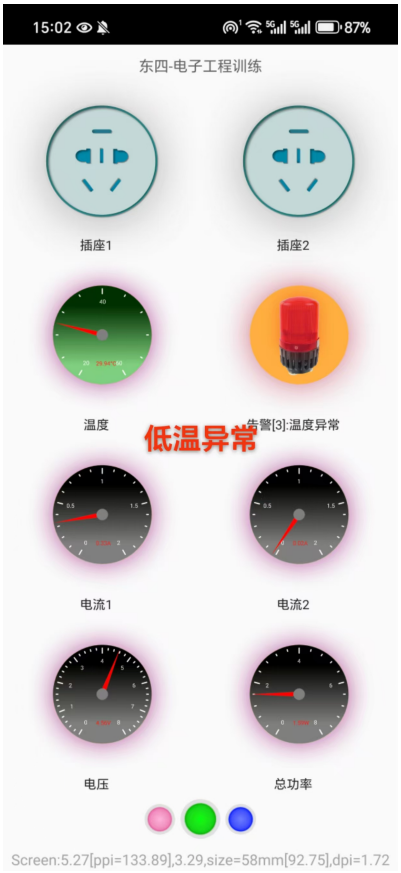


Figure 33: 低温

四、 实验收获与实验总结

1. 实验收获

① 硬件电装与调试能力提升

- 学会了如何焊接多元件的 PCB 板，了解了工程焊接的“焊接-调试”原则，掌握了分模块电装调试的方法
- 学会了如何借助原理图分析电路路径，设计调试方案

② 软硬件联调技能提高

- 学会了通过 Arduino IDE 编写固件代码, 用串口监视器进行通讯
- 学会了软硬件参数标定方法，通过调节可变电阻拟合温度，通过计算公式拟合电流，确保传感器测量值与实际值一致
- 学会了设计移动平均滤波器，有效抑制 PWM 设备电流波动（±20mA→±3mA）
- 了解了按键的抖动原因，学会了通过写消抖函数来减少毛刺带来的按键误识别

③ 掌握系统测试方法

- 学会了系统测试方法，掌握”设计测试用例 → 实践验证 → 修正”的完整测试流程

2. 实验总结

本次实验通过自主焊接、调试、系统测试智能插座的硬件电路，并结合软件编程实现功能控制与数据采集，让我们系统性地掌握了电子产品从需求分析到原型机开发的全流程。重点实践了模块化电装与分步调试的方法、硬件参数标定技术、软硬件协同调试技巧，以及通过测试用例验证系统功能的工程思维。

实验中遇到的虚焊等问题，进一步强化了我们对细节严谨性和工程规范重要性的认识，例如焊接顺序、元器件焊接极性等。通过完整参与研发流程，我们对复杂工程问题的系统性解决能力得到显著提升。

装 订 线